

Reinforcement Learning

2025. 10

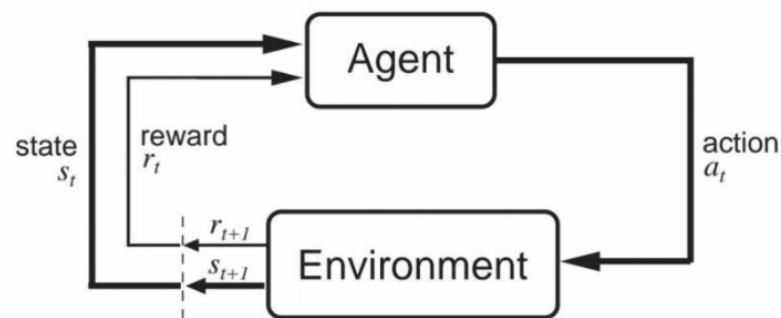
Yongjin Jeong, KwangWoon University

[참고] 본 자료에는 인터넷에서 다운받아 사용한 그림이나 수식들이 일부
있으니 다른 용도로 사용하거나 외부로 유출을 금해 주시기 바랍니다.

Good reference book site (in Korean): <https://dnddnjs.gitbooks.io/rl/>

강화학습 개념과 구조

- 핵심 개념
 - Agent: 학습하는 주체
 - Environment: Agent가 상호작용하는 세계
 - State: 현재 환경의 상태
 - Action: Agent가 선택하는 행동
 - Reward: 행동 결과에 대한 보상
 - Policy: 상태에 따라 행동을 결정하는 전략
- 목표
 - 누적 보상 최대화 (Maximize cumulative reward)



MDP (Markov Decision Process)

- MDP 구성 요소
 - S: 상태 공간 (States)
 - A: 행동 공간 (Actions)
 - $P(s'|s,a)$: 상태 전이 확률 (transition probability)
 - $R(s,a)$: 보상 함수 (Rewards)
 - γ : 할인율 (미래 보상에 대한 중요도) (discount rate)
- 특징
 - 현재 상태와 행동만으로 다음 상태가 결정됨 (Markov 성질)

벨만 방정식

- 가치 함수
 - $V(s)$: 상태의 가치: 상태 s 에서 시작했을 때, 정책 π 를 따를 경우 기대되는 누적 보상 (이 상태가 얼마나 좋은지를 평가)
 - $Q(s,a)$: 상태-행동 쌍의 가치: 상태 s 에서 행동 a 를 취한 후, 정책 π 를 따를 경우 기대되는 누적 보상 (특정 행동이 얼마나 좋은지를 평가)

$$V^\pi(s) = E_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right]$$

$$Q^\pi(s, a) = E_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$$

r_t : t -시점의 보상

- 벨만 기대 방정식

$$Q(s, a) = E[r + \gamma \max_{a'} Q(s', a')]$$

- 의미
 - 현재 행동의 가치는 즉시 보상 + 미래 행동의 가치

Q-learning 알고리즘

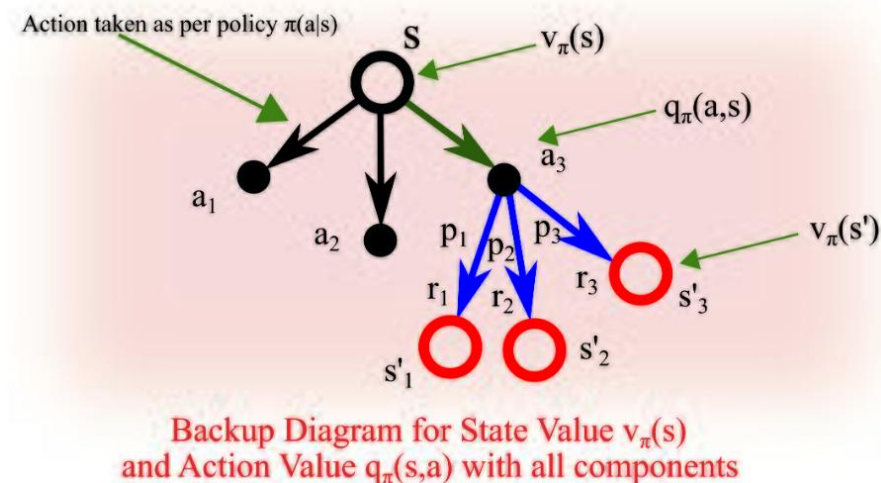
- 특징

- 오프폴리시(Off-Policy) 학습 (현재 정책과 무관하게 학습):
 - 에이전트가 현재 따르고 있는 정책(policy)과는 다른 정책의 데이터를 사용해서 학습하는 방식
 - 학습에 사용하는 행동(action)은 현재 정책이 아닌 다른 정책(예: 과거 정책, 무작위 정책)에서 나온 것
 - 행동은 탐험(exploration) 정책에서 나오지만, 학습은 항상 최적 행동(max Q)을 기준으로 이루어짐.
- Q-table을 통해 상태-행동 가치 저장
- 탐험(Explore) vs. 활용(Exploit) 균형 필요

- 행동 a 와 a' 를 선택하는 다른 정책
- a : 현재 상태 s 에서 실제로 취한 행동 (데이터 수집 시 사용된 정책 → 행동 정책)
- a' : 다음 상태 s' 에서 최적의 행동을 선택 (학습 시 사용되는 정책 → 목표 정책)

- 업데이트 식

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$



DQN (Deep Q-Network)

- 기본 아이디어
 - 기존 Q-learning은 Q-table을 사용했지만, 상태 공간이 크면 불가능 → 대신 신경망(Deep Neural Network)으로 Q값을 근사함.
 - 입력(Input): 현재 상태(State)
 - 출력(Output): 각 행동(Action)에 대한 Q값
 - 목표: 최적 정책을 학습하여 누적 보상을 최대화
- 특징
 - Q-learning + 신경망
 - 경험 재플레이(Replay Buffer)
 - 타겟 네트워크 사용
- 구조
 - CNN or MLP → Q(s,a) 출력
 - 학습 안정성을 위한 기술 적용

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

$$L = \left(r + \gamma \max_{a'} Q_{\text{target}}(s', a') - Q(s, a) \right)^2$$

Why $y = ?$

- 직관적 설명: "미래 보상까지 포함한 가치"
 - 에이전트는 지금 당장 받은 보상만 보면 안 되고, 앞으로 얻게 될 보상까지 고려한 가치(value) 로 판단
 - 그래서 상태-행동 가치 $Q(s,a)$ 의 정의는: "지금 행동해서 바로 받은 보상 + 미래에 받을 보상의 총합"
 - 따라서 당연히 타겟은 다음 상태에서 얻을 수 있는 최대 가치(max Q) 를 포함해야 함.

$$Q(s,a) = r + \gamma \cdot (\text{미래 최고의 } Q)$$

- Bellman Optimality Equation(벨만 최적 방정식)

$$Q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s', a')]$$

- 즉, 최적 Q는 "지금 보상 + 할인된 미래 최적 Q" 관계를 만족해야 한다.

- Q-learning 업데이트

$$Q(s, a) \leftarrow Q(s, a) + \alpha \underbrace{\left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)}_{\text{Q-learning target}}$$

- 괄호 안의 이 값이 타겟이 된다. 즉, Q-learning 타겟 = y .

DQN 주요 기술

- DQN 핵심 개념

- Q-learning을 Deep Neural Network로 근사한 알고리즘
- 경험 재생(Replay Buffer)로 샘플 간 상관성 제거 → 안정화
- Target Network로 Q-learning 업데이트의 불안정성 완화 (Q-learning의 "moving target" 문제 해결 → 안정적 수렴)
- Atari 게임에서 인간 수준 성능 최초 달성
- 핵심 수식은 단 하나: Q-learning target 이용한 MSE 업데이트

- DQN 의 핵심 수식

- Q-Network 함수 근사

$$Q(s, a; \theta)$$

θ : 네트워크 파라미터

θ^- : Target Network 파라미터(고정)

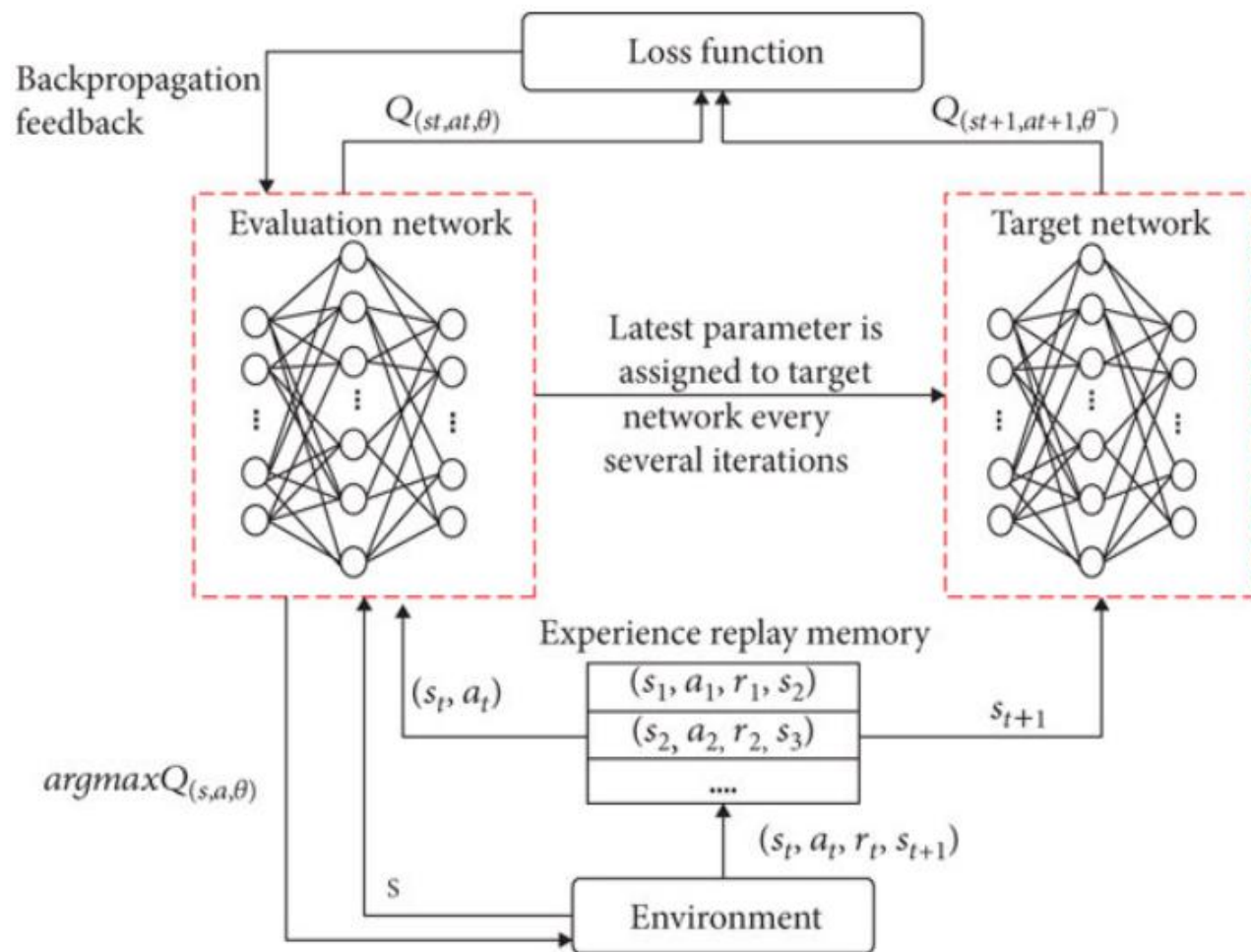
- Target Q-value (강화학습 핵심)

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

- Loss function: MSE

$$L(\theta) = (y - Q(s, a; \theta))^2$$

DQN 주요 기술



Flow chart of the deep Q-learning network.

Ref: Wireless Communications and Mobile Computing, 2020.

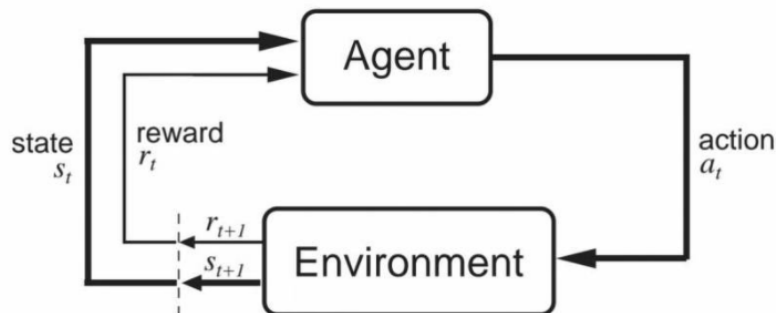
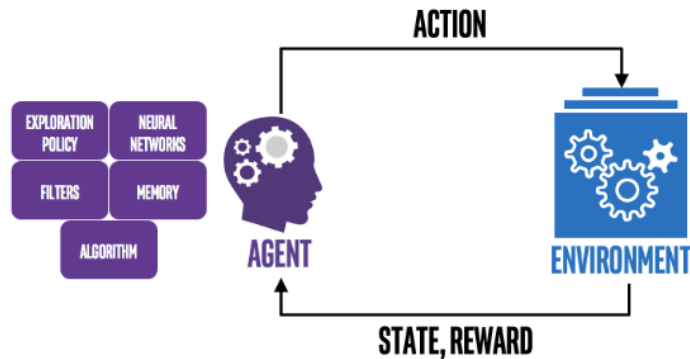
[DQN 동작 흐름]

1. 환경(Environment)에서 현재 상태 s_t 관측
2. Q-Network가 상태를 입력 받아 각 행동의 Q-value 출력
3. 에이전트가 행동 a_t 선택
4. 다음 상태 s_{t+1} 와 보상 r_t 관측
5. 경험을 Replay Buffer에 저장
6. 미니배치로 샘플 $\rightarrow$ Q-Network 업데이트
7. Target Network는 일정 주기마다 Q-Network로부터 복사

Reinforcement Learning

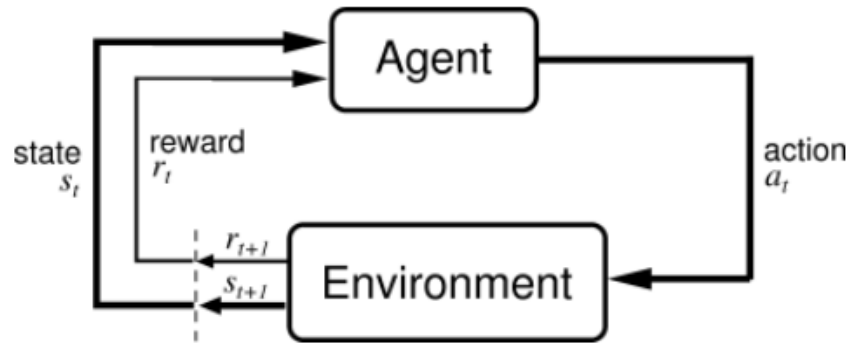
- **Reinforcement Learning (RL)**

- Concerned with sequential decision making process
- Features an agent learning how to act or decide in an environment

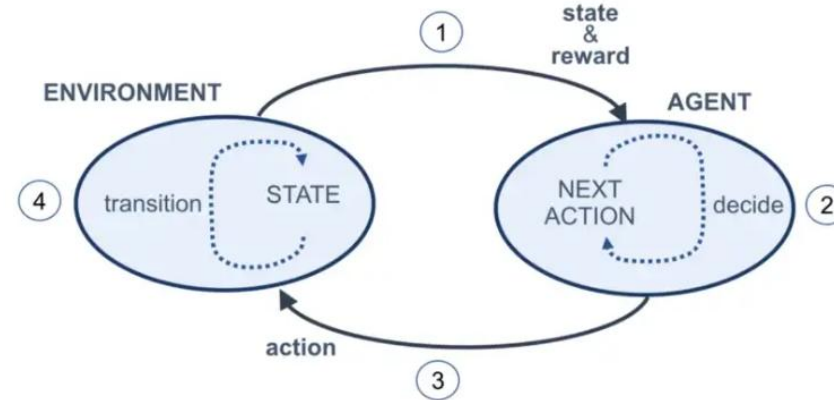


- **Agent** – The program to train (Neural network)
- **Environment** – The world (real or virtual), in which the agent performs actions
- **State** – How the agent sees the environment
- **Action** – A move made by the agent, which causes a status change in the environment
- **Rewards** – The evaluation of an action, which can be positive or negative

Terminologies



Agent-environment interaction [Source]



- **Agent:** It is an assumed entity which performs actions in an environment to gain some reward (robots or human or programs)
- **Environment (e):** A scenario that an agent has to face.
- **Reward (R):** An immediate return given to an agent when he or she performs specific action or task.
- **Action (A):** a set of actions which the agent can perform
- **State (s):** State refers to the current situation returned by the environment.
- **Policy (π):** It is a strategy applied by the agent to decide the next action based on the current state, in other words, a mapping from state s to action a , where $\pi(a/s)$ is the probability of taking action a when in the state s . (*either deterministic or stochastic*)

Terminologies

- **Value (V):** It is expected long-term return with discount
- **Value Function:** It specifies the value of a state that is the total amount of reward, which should be expected beginning from that state.
- **Q value or action value (Q):** Q value is quite similar to value. The only difference between the two is that it takes an additional parameter as a current **action**.
- **Model of the environment:** This mimics the behavior of the environment. By a model of the environment, we mean **a function that predicts state transitions and rewards**.
- **Model-Free vs. Model-Based RL**
 - **Model-based:** after learning, the agent can make predictions about what the next state and reward will be before it takes each action. **Dynamic Programming (policy iteration and value iteration)** use the model's predictions or distributions of next state and reward in order to calculate optimal actions.
 - **Model-free:** no knowledge of MDP transitions and rewards - a ground-truth model of the environment is usually not available to the agent. If an agent wants to use a model, it has to learn the model **purely from experience**. **Monte Carlo, Temporal Difference (TD), SARSA, Q-learning, Actor-Critic** rely on real samples from the environment.

RL Methods

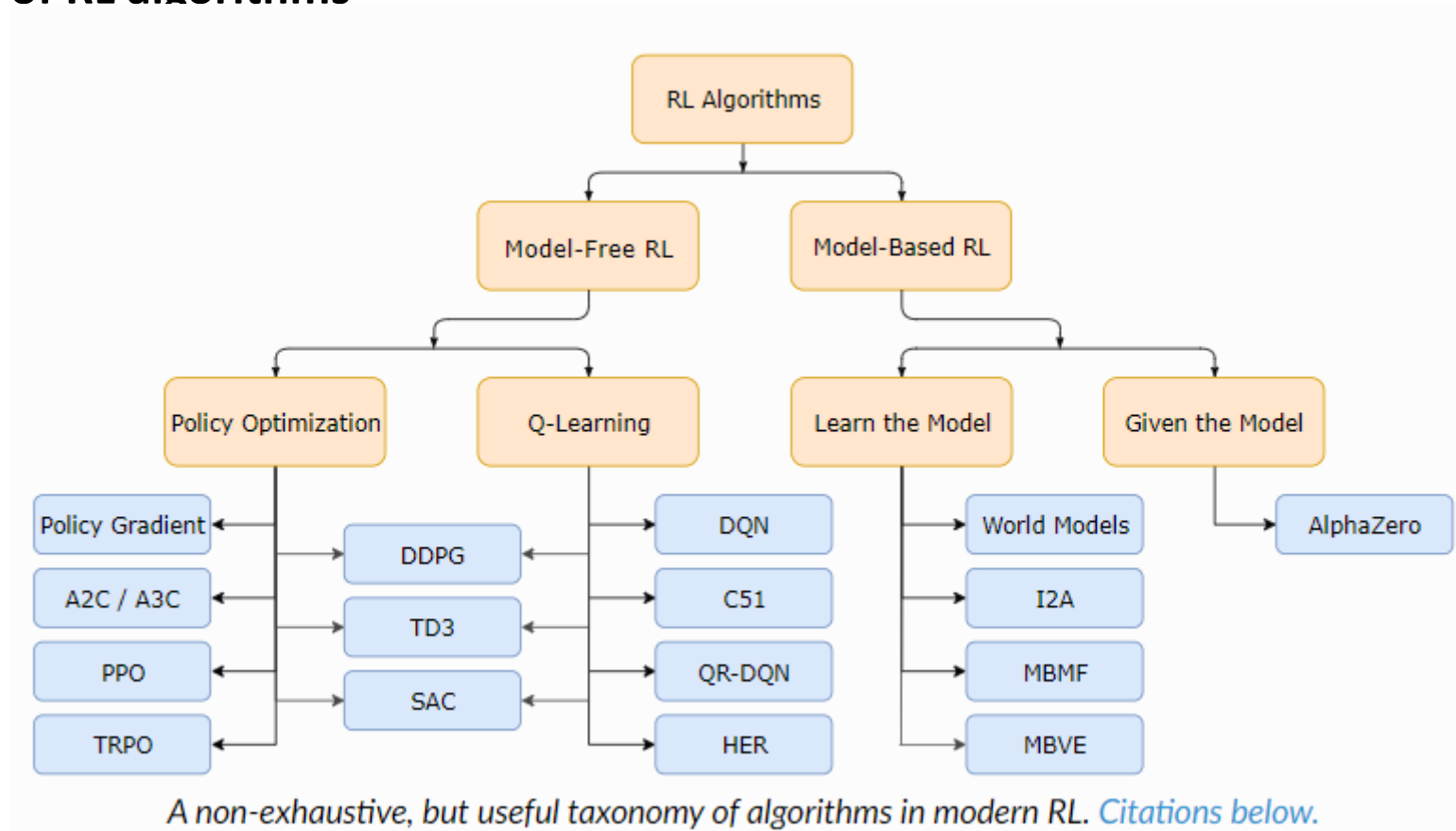
- **Dynamic programming (DP)** — good starting point to understand RL algorithms that can solve more complex problems
- **Monte Carlo methods** — A class of methods for learning of value functions, which estimates the value of a state by running many trials starting at that state, then averages the total rewards received on those trials. (episode $\square \square \square$ update)
- **Temporal Difference (TD) algorithms** — A class of learning methods, based on the idea of comparing temporally successive predictions. Possibly the single most fundamental idea in all of reinforcement learning. (time step $\square \square \square$ update)
- **What to Learn?**
 - **policies**, either stochastic or deterministic,
 - **action-value functions** (Q-functions),
 - **value functions**, (V)
 - and/or environment models.

RL Methods (from GPT)

- 동적 프로그래밍 (Dynamic Programming, DP)
 - 환경 모델 필요: 상태 전이 확률과 보상 함수 등 환경의 완전한 정보(모델)가 있어야 사용 가능.
 - 벨만 방정식 활용: 가치 함수나 정책을 벨만 방정식으로 반복적으로 업데이트.
 - 정확한 계산: 모든 상태에 대해 가치 함수를 계산하기 때문에 이론적으로 최적의 정책을 보장.
 - 탐색 방식: 전체 상태 공간을 반복적으로 업데이트 (모든 상태를 고려).
 - 이론적으로 강력하며 최적해를 보장하지만 상태 공간이 크면 계산량이 급격히 증가(차원의 저주).
- 몬테카를로 방법 (Monte Carlo, MC)
 - 환경 모델 불필요: 샘플 데이터를 통해 학습하므로 환경의 사전 정보가 없어도 사용 가능.
 - 에피소드 단위 학습: 전체 에피소드가 끝난 후 누적 보상을 계산해 업데이트.
 - 무작위 샘플링: 시뮬레이션을 통해 데이터를 수집.
 - 탐색 방식: 환경의 모든 상태를 탐색할 필요는 없으며, 경험을 통해 접근.
 - 모델 프리로 작동하며 환경이 복잡해도 적용 가능하지만 학습이 에피소드 종료에 의존하므로 학습 속도가 느림.
- TD 방법 (Temporal Difference)
 - 환경 모델 불필요: 샘플 데이터를 사용하여 학습하며, 환경 모델이 없어도 사용 가능.
 - 부트스트래핑 사용: 벨만 방정식처럼 미래 가치 추정치를 기반으로 현재 상태를 업데이트.
 - 점진적 학습: 에피소드가 끝나기 전에 데이터가 들어올 때마다 가치 함수를 업데이트.
 - 탐색 방식: 몬테카를로처럼 데이터를 활용하되, 동적 프로그래밍처럼 미래 가치 추정치를 사용.
 - 효율적이고 빠르게 학습 가능하지만 가치 추정치에 의존하므로 초기 학습 과정에서 오류 가능성.
 - 예시: SARSA, Q-learning

Taxonomy

- [OpenAI](https://spinningup.openai.com/en/latest/index.html) — a non-profit AI research company with the mission to build and share safe Artificial General Intelligence (AGI) — launched a program to “spin up” deep RL.
(<https://spinningup.openai.com/en/latest/index.html>)
- **Taxonomy of RL algorithms**



Real Life Applications

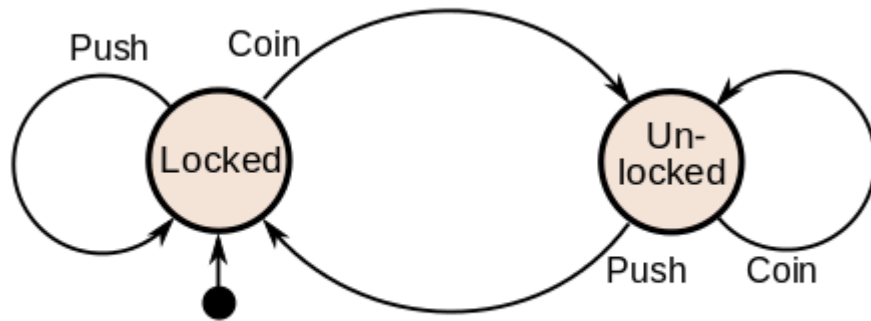
- **Self-driving cars:** trajectory optimization, motion planning, dynamic pathing, controller optimization, and scenario-based learning policies for highways.
- **Industry automation:** energy saving (Google), industrial robots
- **Trading and finance:** better stock exchange than time series model (supervised learning)
- **NLP:** text summarization, question answering, machine translation
- **healthcare:** medical treatment options for every disease stage
- **engineering:** optimize video streaming quality based on the state of video buffer, etc.
- **News recommendation:** track reader's return behavior
- **Gaming:** AlphaGo Zero
- **Marketing and advertisement:** real-time bidding with multi-agent RL
- **Robotics manipulation:** Boston Dynamics robots
- See <https://neptune.ai/blog/reinforcement-learning-applications>

Reinforcement Learning Algorithms

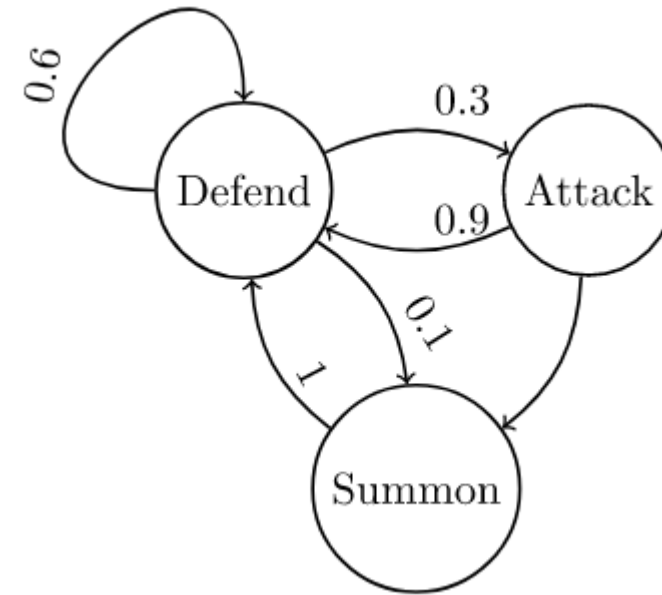
- **Value-based:** aim to find the optimal value function, often denoted as " $Q(s, a)$ " or " $V(s)$,"
 - Once the optimal value function is learned, the optimal policy can be derived directly from it.
 - (ex) Q-learning, Deep Q-Networks (DQN), SARSA
 - It is effective in environments with large state and action spaces. They are also known for their stability and ease of convergence.
- **Policy-based:** optimize the policy function " $\pi(a|s)$ " directly
 - It directly parameterize the policy itself, which is a mapping from states to actions. This can be represented as " $\pi(a|s)$," where π is the policy function.
 - Deterministic: For any state, the same action is produced by the policy π .
 - Stochastic : Every action has a certain probability.
 - (ex) REINFORCE
 - more suitable for problems with high-dimensional action spaces and complex, continuous action domains.
- **Model-based**
 - You need to create a virtual model for each environment. The agent learns to perform in that specific environment.

Finite State Machine and Markov Chain

- Finite State Machine (FSM) – Deterministic
- Markov Chain – Stochastic (Probabilistic, Random)

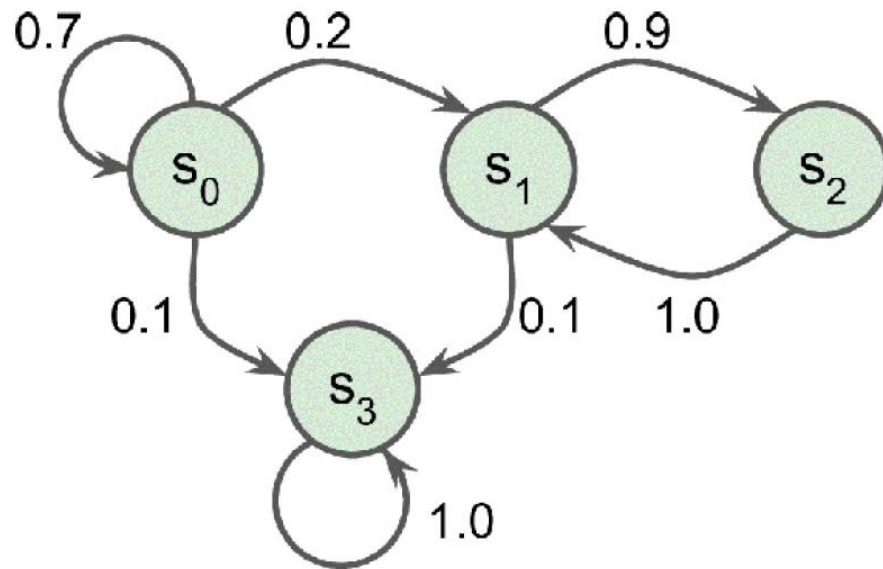


State diagram for a turnstile



Markov Chain (or Markov Process)

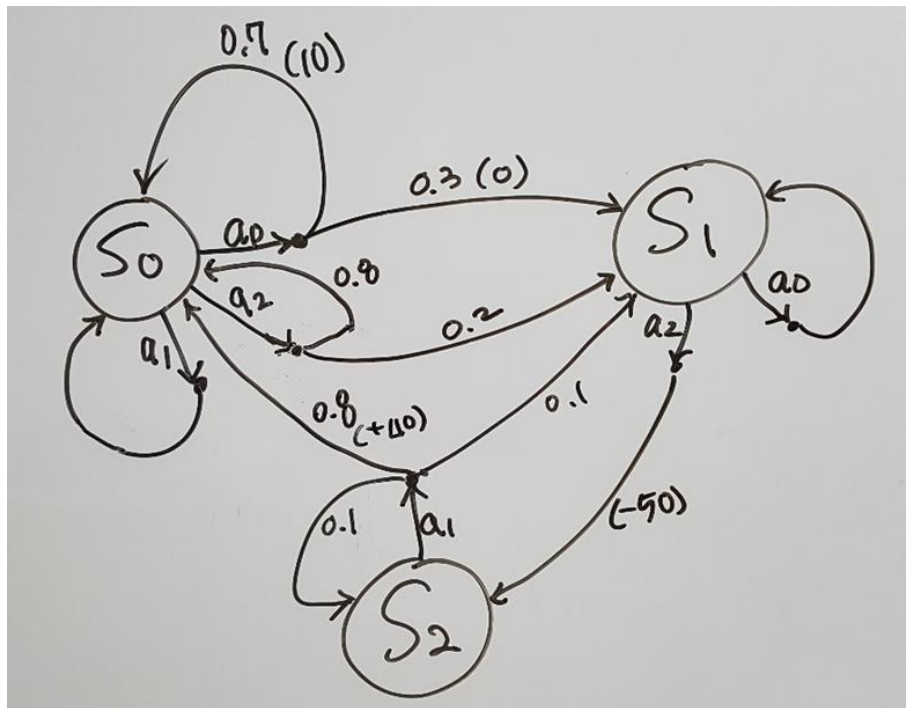
- Stochastic processes with no memory
- Has a **fixed number of states**
- **Probability** from a state s to s' is fixed, and it depends only on the pair (s,s') , not on the past state (the system has no memory)
- **Markov Process**: continuous-time version of a Markov chain



```
transition_probabilities = [  
    [0.7, 0.2, 0.0, 0.1],  
    [0.0, 0.0, 0.9, 0.1],  
    [0.0, 1.0, 0.0, 0.0],  
    [0.0, 0.0, 0.0, 1.0]]
```

Markov Decision Processes (MDP)

- They resemble Markov chains but with a twist: at each step, an agent can choose one of **several possible actions**, and
- The **state transition probabilities** depend on the chosen **action**. Moreover, some state transitions return some **reward** (positive or negative), and
- The agent's goal is to find a policy that will maximize rewards over time.



```
transition_probabilities = [
    [[0.7, 0.3, 0.0],
     [1.0, 0.0, 0.0],
     [0.8, 0.2, 0.0]],

    [[0.0, 1.0, 0.0],
     None,
     [0.0, 0.0, 1.0]],

    [None,
     [0.8, 0.1, 0.1],
     None]]
```

```
rewards = [ # shape=[s, a, s']
    [[+10, 0, 0],
     [0, 0, 0],
     [0, 0, 0]],

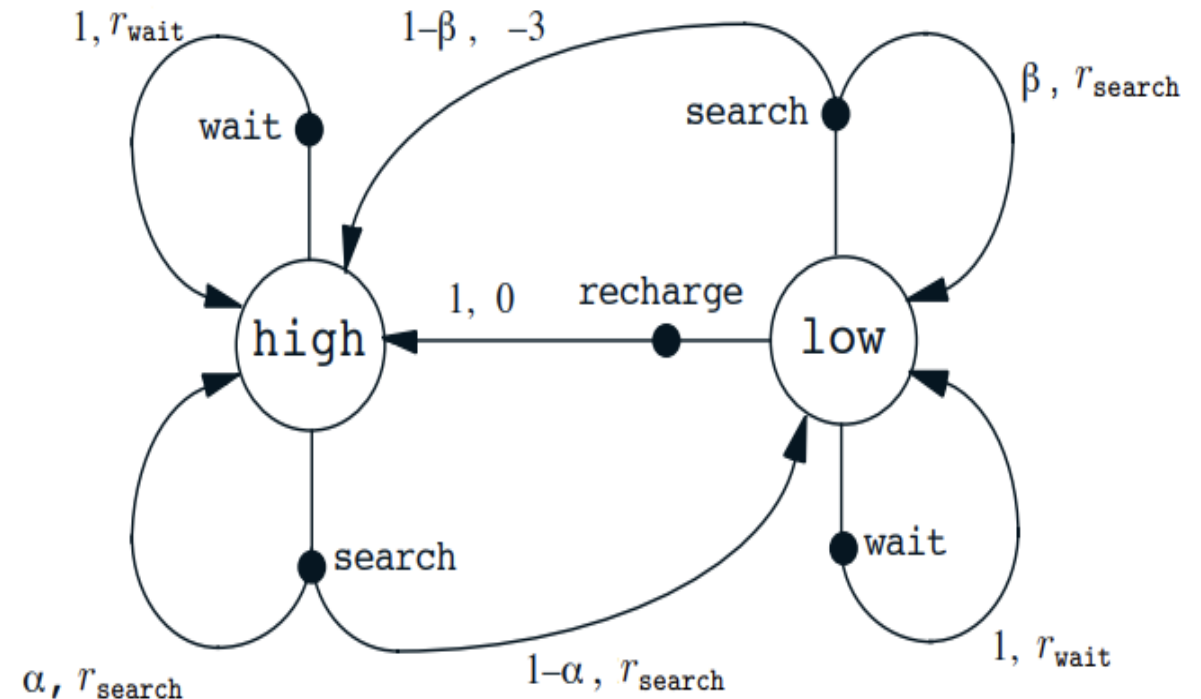
    [[0, 0, 0],
     [0, 0, 0],
     [0, 0, -50]],

    [[0, 0, 0],
     [+40, 0, 0],
     [0, 0, 0]]]
```

```
possible_actions = [[0, 1, 2], [0, 2], [1]]
```

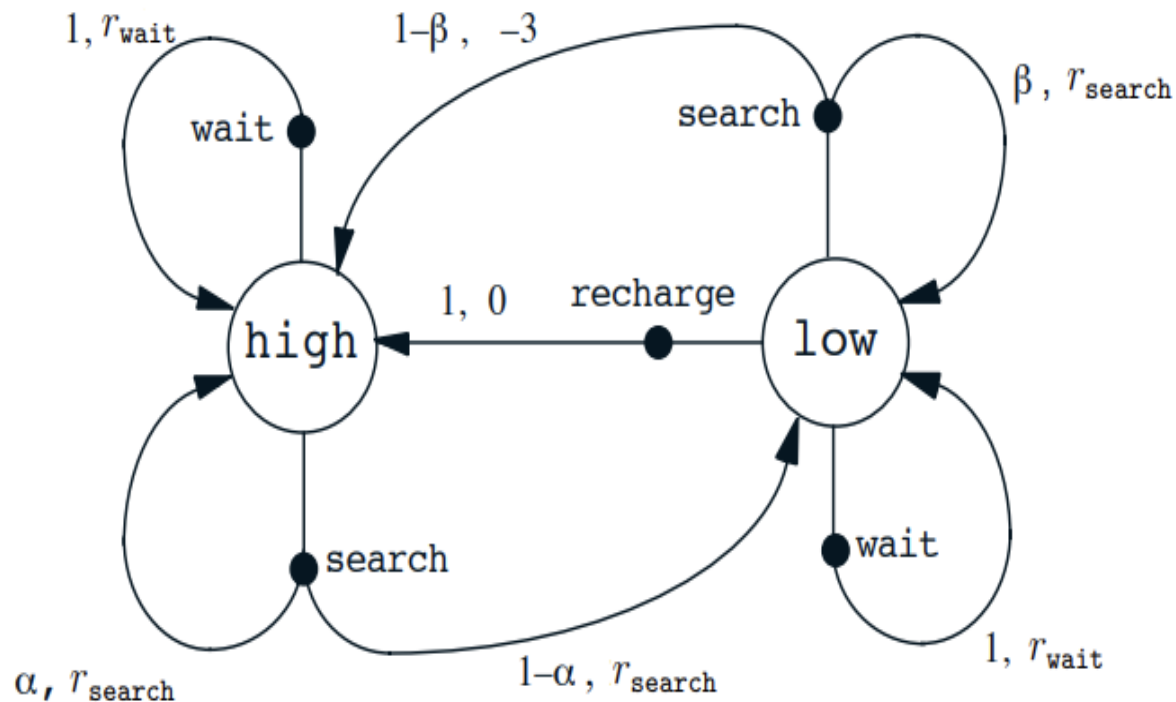
Example: Recycling Robot MDP

- **State:** Energy level of the battery, $S = \{\text{High}, \text{Low}\}$
- **Action:**
 - $A(\text{high}) = \{\text{search}, \text{wait}\}$
 - $A(\text{low}) = \{\text{search}, \text{wait}, \text{recharge}\}$
- A transition graph:
 - Two kinds of nodes: **state nodes** and **action nodes**.
 - There is a state node for each possible state (a large open circle labeled by the name of the state)
 - an action node for each state-action pair (a small solid circle labeled by the name of the action and connected by a line to the state node)



Transition graph (Sutton and Barto)

Example: Recycling Robot MDP

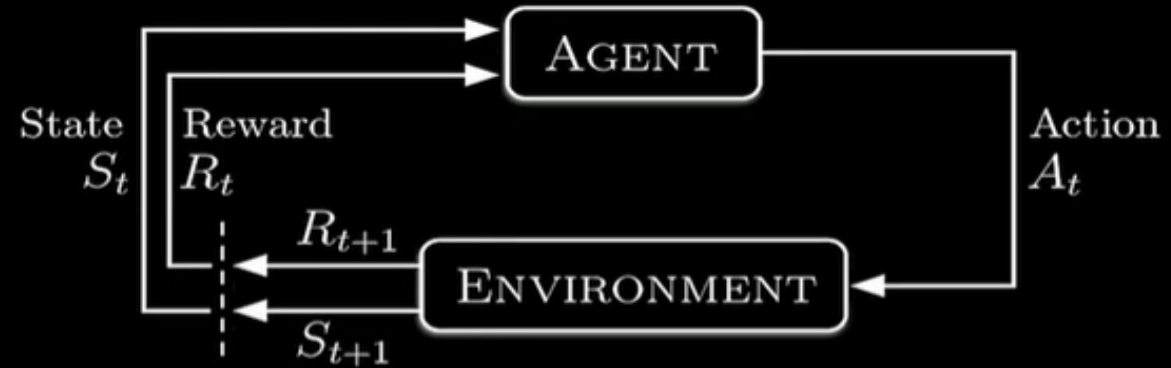


s	s'	a	$p(s' s, a)$	$r(s, a, s')$
high	high	search	α	r_{search}
high	low	search	$1 - \alpha$	r_{search}
low	high	search	$1 - \beta$	-3
low	low	search	β	r_{search}
high	high	wait	1	r_{wait}
high	low	wait	0	r_{wait}
low	high	wait	0	r_{wait}
low	low	wait	1	r_{wait}
low	high	recharge	1	0
low	low	recharge	0	0.

Transition graph (Sutton and Barto)

RL Model

Finite Markov Decision Process



$$t \in \{0, 1, 2, \dots\} \quad s \in \mathcal{S} \quad a \in \mathcal{A}(s) \quad r \in \mathcal{R} \in \mathbb{R}$$

$$p(s', r | s, a) = \text{Prob}(S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a)$$

$$\text{POLICY: } \pi(a|s) \quad \text{or} \quad a = \pi(s)$$

$$\text{RETURN: } G_t = \sum_{k=t+1}^T \gamma^{k-t-1} R_k$$

$$\text{GOAL: } \max_{\pi} \mathbb{E}_{\pi}[G_t]$$

Returns

- **Agent's Goal:**

- To maximize the cumulative reward it receives in the long run ([to find the optimal policy](#))

- **Returns**

- For the sequence of rewards received after time step t : $R_{t+1} R_{t+2} R_{t+3}, \dots$
- **Return** G_t : sum of the rewards (T is a final time step)

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

- $t = 1, 2, \dots, T$ (the sequences are called [episodes](#).)
- Each episode ends in a special state called the terminal state. ([episodic task](#))
- $t = 1, 2, \dots, T=\infty$ ([continuing task](#))

- **Discounting**

- determines the present value of future rewards (a reward received at $t=k$ time steps in the future is worth only γ^{k-1} times what it would be worth if it were received immediately. (discount rate γ : $0 \leq \gamma \leq 1$)
- **Discounted return**, G_t :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Value Functions

- **Value Functions**

- functions of states (or of state–action pairs) that estimate **how good it is for the agent to be in a given state** (or **how good it is to perform a given action in a given state**).
- The notion of “how good” here is defined in terms of future rewards that can be expected, (or, in terms of expected return, G_t)
- Of course, the rewards the agent can expect to receive in the future depend on what actions it will take. Accordingly, value functions are defined for particular policies.

(*) note that the value of the terminal state, if any, is always zero.

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s\right]$$

state-value function for policy π

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a\right]$$

action-value function for policy π

Policy and Value Functions

- **Policy:** probability distribution over actions

$$\text{policy} = \pi(a|s) = P(A_t=a | S_t=s)$$

- **State-Value function, $v_\pi(s)$**

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

"return": t 시점 이후의 모든 discount 된 reward 의 총합

$$v_\pi(s) = \mathbb{E}_\pi [G_t | S_t = s]$$

$$= \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s \right]$$

$$= \mathbb{E}_\pi [R_{t+1} + \gamma G_{t+1} | S_t = s]$$

Value(가치) of state s

- **Action-Value function (Q-Function)**

$$q_\pi(s, a) = \mathbb{E}_\pi [G_t | S_t = s, A_t = a]$$

$$= \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s, A_t = a \right]$$

$$= \mathbb{E}_\pi [R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a]$$

Value (가치) of state-action (s,a)

Formal Definitions

Definition

A *Markov Decision Process* is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{A}$ is a finite set of actions
- $\mathcal{P}$ is a state transition probability matrix,
 $\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$
- $\mathcal{R}$ is a reward function, $\mathcal{R}_s^a = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$
- γ is a discount factor $\gamma \in [0, 1]$.

Definition

A *policy* π is a distribution over actions given states,

$$\pi(a|s) = \mathbb{P}[A_t = a \mid S_t = s]$$

Definition

The *return* G_t is the total discounted reward from time-step t .

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Definition

The *state-value function* $v_{\pi}(s)$ of an MDP is the expected return starting from state s , and then following policy π

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

Definition

The *action-value function* $q_{\pi}(s, a)$ is the expected return starting from state s , taking action a , and then following policy π

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

Bellman Equation

- 강화학습에서는 어떤 행동(Action)에 대한 보상(Reward)이 필요하다.
 - 이 때 보상(Reward)은 현재 **당장 받는 보상 (immediate reward)** 뿐만 아니라 **미래의 보상 (future reward)**을 함께 고려해야 한다.
- 강화학습에서 미래의 보상을 알기 위해 앞으로 일어날 일을 모두 경험하지 않더라도 현재의 상태에서 예측을 통해서 미래를 가늠해 볼 방법을 이용한다. **벨만 방정식(Bellman Equation)**은 이처럼 현재 상태에서부터 미래의 성공확률을 계산하기 위한 식이다.
- 가치함수(Value Function)은 다음의 두 개의 부분으로 구성된다.
 - 행동 후 즉각 받는 보상 (immediate reward)
 - 감가율을 고려한 미래상태 (future state)에 대한 가치 (value)

$$\begin{aligned}v(s) &= \mathbb{E}[G_t \mid S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \dots) \mid S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s]\end{aligned}$$

immediate reward

next state value

; linearity of expectation

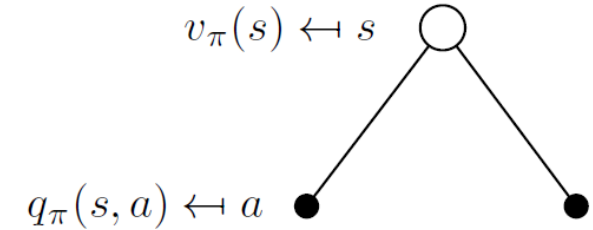
$$E(x + \gamma E(y)) = E(x) + \gamma E(y)$$

constant

Bellman (expectation) Equation

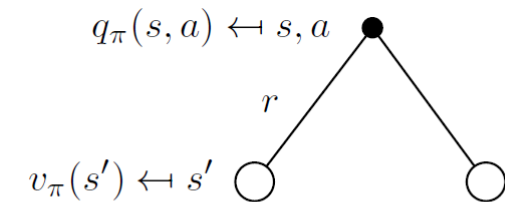
- 상태가치 (State value)에 대한 Bellman 방정식
 - 현재의 상태와 뒤이어질 상태들의 모든 가치를 계산

$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s]$$



- 행동가치 (Action value)에 대한 Bellman 방정식
 - 현재 상태(State)에서 어떤 정책(Policy)에 따라 행동 (Action)을 취했을 때 뒤이어질 상태(next State)와 행동(Action)들에 따라 얻게 되는 가치를 계산

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} [R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

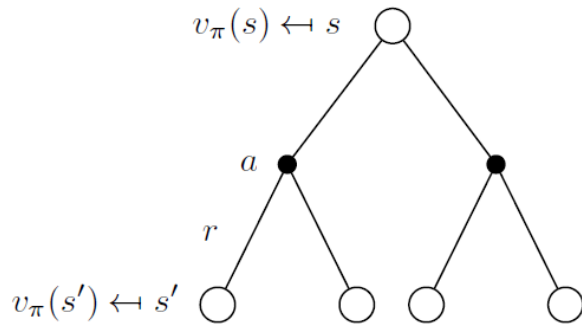


$$q_{\pi}(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_{\pi}(s')$$

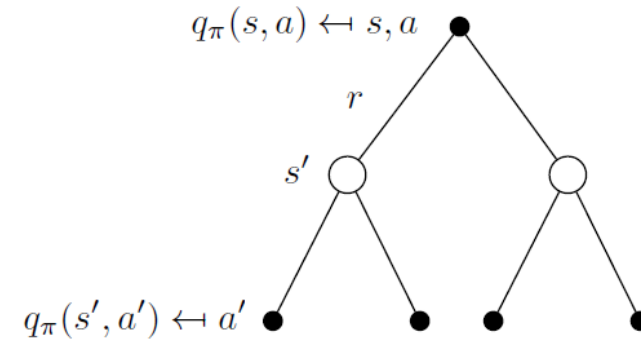
(*) Note that $R(s)$ or $R(s,a)$ is often used in the Bellman equations for brevity, but this is a simplified representation of the more detailed $R(s,a,s')$ in the MDP framework.

Bellman (expectation) Equation

- 앞의 두그림을 합치면,



$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s') \right)$$



$$q_\pi(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') q_\pi(s', a')$$

Bellman Equation

$$\begin{aligned}
 v_{\pi}(s) &= \mathbb{E}_{\pi} [G_t | S_t = s] \\
 &= \mathbb{E}_{\pi} [R_{t+1} + \gamma G_{t+1} | S_t = s] \\
 &= \mathbb{E}_{\pi} [R_{t+1} + \gamma \mathbb{E}_{\pi} [G_{t+1} | S_{t+1} = s'] | S_t = s] \\
 &= \sum_a \pi(a|s) \sum_{s', r} \mathbb{P}(s', r | s, a) [r + \gamma \mathbb{E}_{\pi} [G_{t+1} | S_{t+1} = s']] \\
 &= \sum_a \pi(a|s) \sum_{s', r} \mathbb{P}(s', r | s, a) [r + \gamma v_{\pi}(s')] \\
 &= \sum_a \pi(a|s) q_{\pi}(s, a)
 \end{aligned}$$

$$\begin{aligned}
 q_{\pi}(s, a) &= \mathbb{E}_{\pi} [G_t | S_t = s, A_t = a] \\
 &= \mathbb{E}_{\pi} [R_{t+1} + \gamma \mathbb{E}_{\pi} (G_{t+1} | S_{t+1} = s') | S_t = s, A_t = a] \\
 &\quad \text{or } \mathbb{E}_{\pi} [R_{t+1} + \gamma \mathbb{E} (G_{t+1} | S_{t+1} = s', A_{t+1} = a') | S_t = s, A_t = a] \\
 &= \mathbb{E} [R_{t+1} + \gamma v_{\pi}(s') | S_t = s, A_t = a] \\
 &\quad \text{or } \mathbb{E} [R_{t+1} + \gamma q_{\pi}(s', a') | S_t = s, A_t = a] \\
 &= \sum_{r, s'} \mathbb{P}(s', r | s, a) [r + \gamma v_{\pi}(s')] \\
 &\quad \text{or } \sum_{r, s', a'} \mathbb{P}(r, s', a' | s, a) [r + \gamma q_{\pi}(s', a')]
 \end{aligned}$$

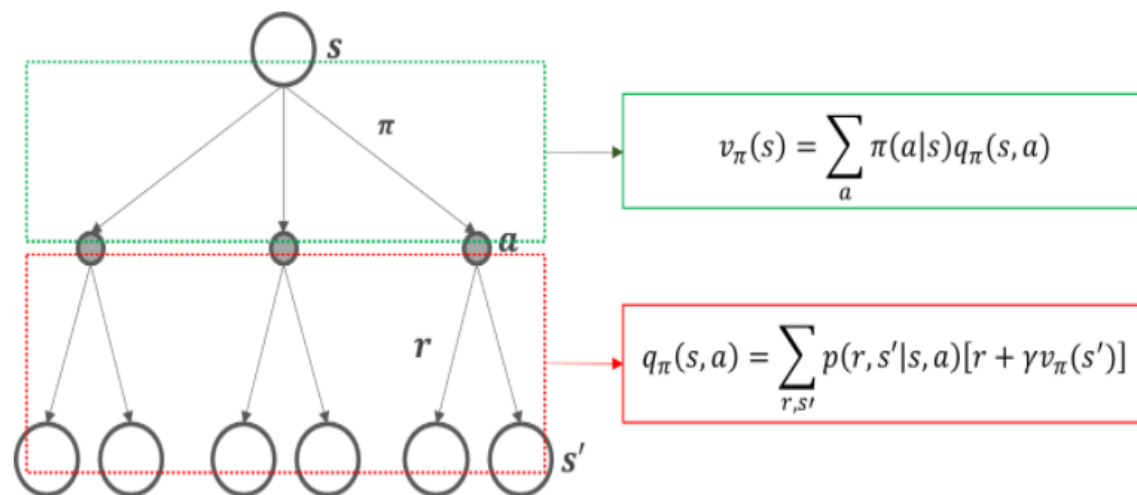


Figure 2: Bellman Equation Back-up Diagram

Bellman Optimality Equation

- 강화학습의 목적은 가치함수의 참값을 찾는 것이 아니라 **최대의 보상(Reward)를 얻는 정책(Policy)를 찾는 것**이다. 즉, 가치함수의 가장 **큰** 참값을 찾는 것이다. 이와 같이 최적의 정책을 구하고자 하는 벨만방정식을 Bellman Optimality Equation이라고 한다.

- Optimal policy, π^* :

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \operatorname{argmax}_{a \in \mathcal{A}} q_*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

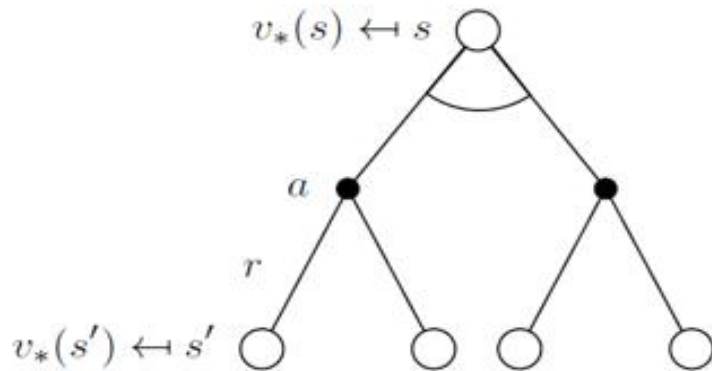
- 최적가치함수(Optimal Value Function)

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')] \quad \text{for all } s$$

- $T(s, a, s')$ is the transition probability from state s to state s' , given that the agent chose action a .
- $R(s, a, s')$ is the reward that the agent gets when it goes from state s to state s' , given that the agent chose action a .
- γ is the discount rate.

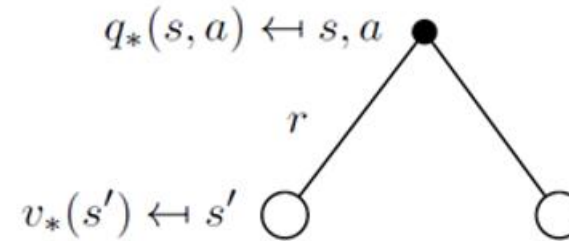
Bellman Optimality Equation

- 최적상태가치함수(Optimal State-Value Function)



$$\begin{aligned}
 V_*(s) &= \max_{\pi} V_{\pi}(s) = \max_a q_{\pi}(s, a) \\
 &= \max_a \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a V_{\pi}(s') \right) \\
 &= \max_a R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a V_*(s')
 \end{aligned}$$

- 최적행동가치함수(Optimal Action-Value Function)



$$\begin{aligned}
 q_*(s, a) &= \max_{\pi} q_{\pi}(s, a) = \max_{\pi} \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a V_{\pi}(s') \right) \\
 &= R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a V_*(s')
 \end{aligned}$$

Bellman Optimality Equation

1. Bellman Expectation Function:

- The Bellman expectation function is associated with both state-value functions $V(s)$ and action-value functions $Q(s, a)$.
- It expresses the expected value of a state or state-action pair in terms of the immediate reward and the expected value of the successor states under a given policy.
- For the state-value function, the Bellman expectation equation is given by:

$$V(s) = \sum_a \pi(a|s) (R(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s'))$$

- For the action-value function, it is expressed as:
$$Q(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \sum_{a'} \pi(a'|s') Q(s', a')$$
- Here, $\pi(a|s)$ represents the policy, indicating the probability of taking action a in state s .

2. Bellman Optimality Function:

- The Bellman optimality function is specifically associated with optimal policies and the optimal value functions.
- It describes the relationship between the optimal value of a state or state-action pair and the optimal value of its successor states.
- For the state-value function, the Bellman optimality equation is given by:

$$V^*(s) = \max_a (R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s'))$$

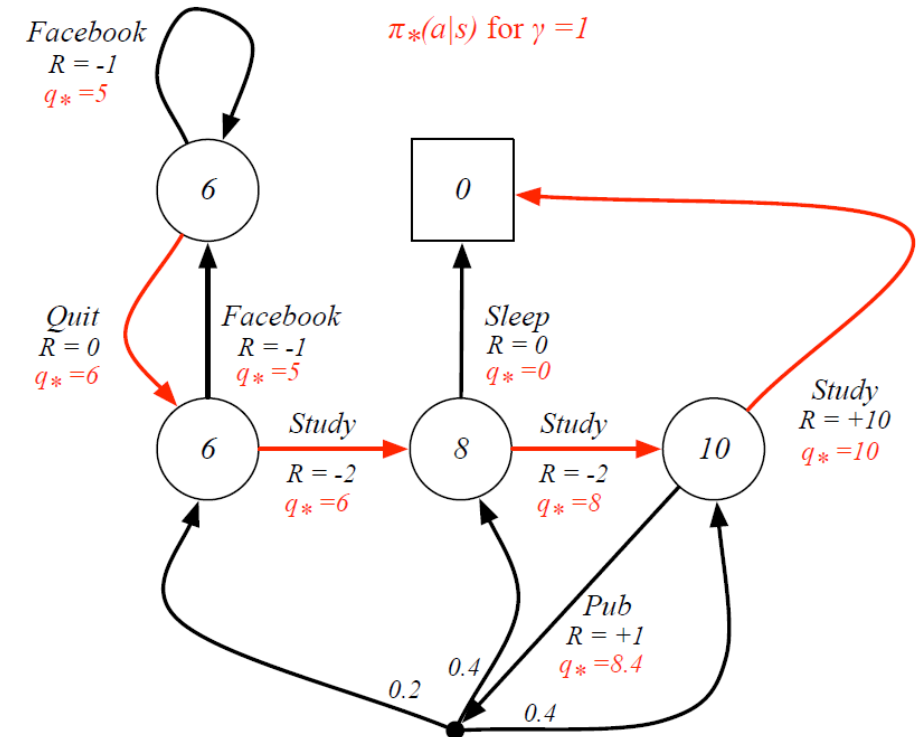
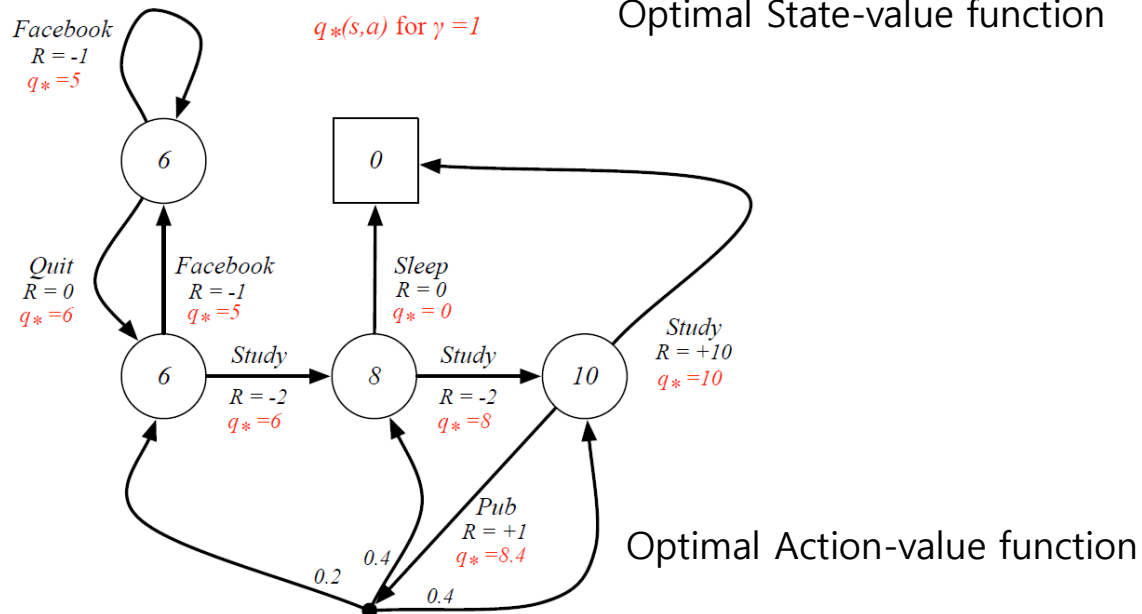
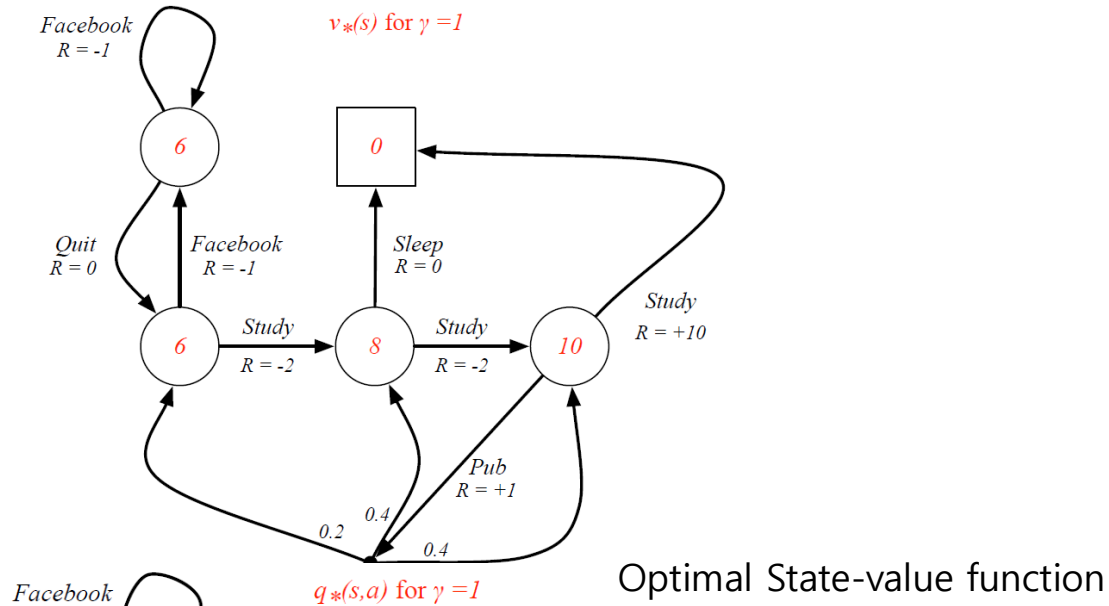
- For the action-value function, it is expressed as:
$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a')$$
- Here, $V^*(s)$ and $Q^*(s, a)$ represent the optimal state-value and action-value functions, respectively.

(*) 즉, 벨만기대함수는 주어진 정책 하에서 기대값으로 표현되고, 벨만최적함수는 가능한 최선의 정책 하에서 최적의 값(최대값)으로 표현됨. Bellman 최적 방정식은 강화 학습에서 최적의 정책을 찾는 데 기본이다.

Bellman Optimality Equation (example)

Optimal policy: take an action that maximizes values

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \operatorname{argmax}_{a \in \mathcal{A}} q_*(s, a) \\ 0 & \text{otherwise} \end{cases}$$



Reinforcement Learning Algorithms

- Dynamic Programming
 - (model-based)
 - Not a Reinforcement Learning, but a **Planning** (in the sense that it assumes complete knowledge about the environment)
 - Policy iteration: alternates between two steps: policy evaluation and policy improvement)
 - Value iteration: iteratively computes and improves the value function for each state until it converges to the optimal value function. The policy is then derived from the optimal value function
- Monte Carlo Learning
- Temporal Difference Learning:
 - SARSA
 - Q-Learning

DP and RL

특징	Dynamic Programming (DP)	Reinforcement Learning (RL)
문제 해결 방식	작은 문제의 해를 저장하고 이를 조합하여 큰 문제를 해결	중간 단계의 값(예: Q-값)을 반복적으로 업데이트하며 최적 정책 학습
테이블 사용	상태의 해를 테이블에 저장	테이블(예: Q-값)을 학습 과정에서 점진적으로 수정
환경 모델	환경 모델(전이 확률 및 보상 함수)이 주어져야 함	환경 모델이 없어도 학습 가능 (모델 프리)
정적/동적 업데이트	테이블 값은 정적으로 계산되고 수정되지 않음	학습 중 계속 업데이트
계산 방식	정확한 수치 계산 (정확한 모델 필요)	근사 계산 (샘플 기반 학습)
최적화 목표	항상 최적의 해를 계산	탐험과 활용 균형을 통해 최적 정책에 수렴

비유: DP vs RL

- DP는 퍼즐 조각을 하나씩 조합해 완성된 그림을 만드는 과정과 같습니다.
- RL은 희미한 그림을 점진적으로 수정하며 최종 그림을 학습하는 과정과 비슷합니다.

Reinforcement Learning Algorithms

- Policy iteration
 - obtain a sequence of monotonically improving policies and value functions: (policy evaluation and policy improvement)

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*$$

1. Initialization
 $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$
2. Policy Evaluation
Repeat
 $\Delta \leftarrow 0$
 For each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{s',r} p(s', r | s, \pi(s)) [r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$ (a small positive number)
3. Policy Improvement
 policy-stable $\leftarrow$ true
 For each $s \in \mathcal{S}$:
 $a \leftarrow \pi(s)$
 $\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$
 If $a \neq \pi(s)$, then *policy-stable* $\leftarrow$ false
 If *policy-stable*, then stop and return V and π ; else go to 2

Reinforcement Learning Algorithms

- Value iteration
 - Special case of policy iteration: when policy evaluation is stopped after just one sweep (one backup of each state) : 벨만 최적 방정식을 업데이트 형식으로 바뀐 것

$$\begin{aligned}v_{k+1}(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s, A_t = a] \\&= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_k(s')],\end{aligned}$$



$$\begin{aligned}v'_\pi(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v'_\pi(S_{t+1}) \mid S_t = s, A_t = a] \\&= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v'_\pi(s')]\end{aligned}$$

Bellman optimality equation

Initialize array V arbitrarily (e.g., $V(s) = 0$ for all $s \in \mathcal{S}^+$)

Repeat

$\Delta \leftarrow 0$

For each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number)

Output a deterministic policy, π , such that

$\pi(s) = \operatorname{argmax}_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma V(s')]$

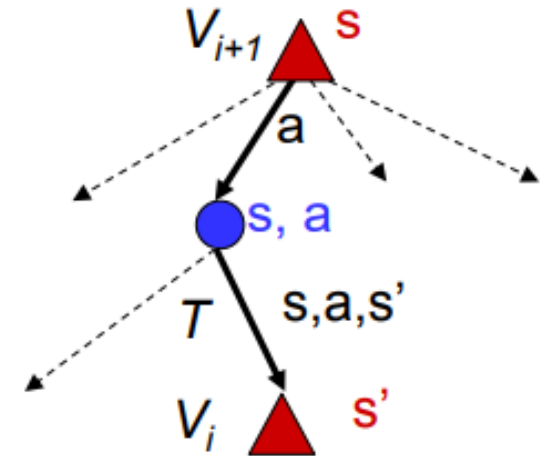
State-Value Iteration algorithm

- An algorithm to precisely estimate the optimal state value of every possible state:
 - Initialize all the state value estimates: $V_0(s) = 0$
 - Iteratively update them using **Value Iteration algorithm**

$$V_{i+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_i(s')]$$

- Equivalent to: $V_{i+1}(s) \leftarrow \max_a Q_{i+1}(s, a)$

- Q: why not directly Q-values? -> Q-value iteration



Q-Value Iteration Algorithm

- Knowing the optimal state values can be useful (to evaluate a policy), but it does **not tell the agent explicitly what to do**.
- A very similar algorithm to estimate **the optimal state-action values (Q-values) of the state-action pair (s,a)** , noted $Q^*(s,a)$:

$$Q_*(s, a) = \max_{\pi} Q_{\pi}(s, a)$$

- **Precise estimation:**
 - Initialize all the Q-value estimates = 0
 - Iteratively update them using **Q-value Iteration algorithm**

$$Q_{i+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma V_i(s') \right] \text{ i.e.,}$$
$$Q_{i+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma \max_{a'} Q_i(s', a') \right]$$

- Once you have the optimal Q-values, the optimal policy $\pi^*(s)$ is trivial:

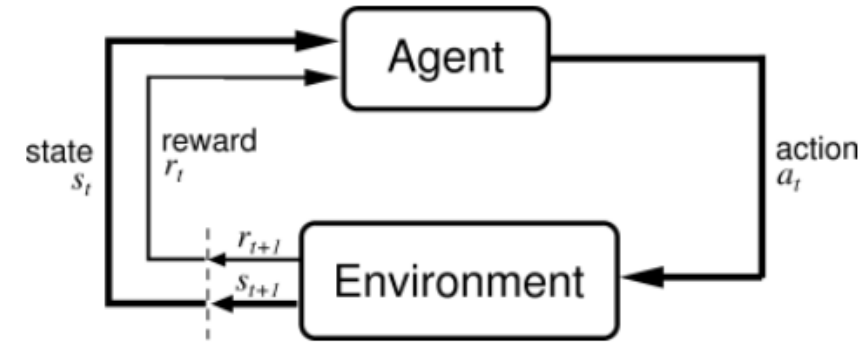
$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

Q-iteration -> Q-learning

$$Q_{i+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma \max_{a'} Q_i(s', a') \right]$$

In RL, we don't have this!

- But we get a sample at each time step $t: (s_t, a_t, r_{t+1}, s_{t+1})$



Agent-environment interaction [Source]

- Instead of Expectation under T , what if we compute a running average of Q from all samples received so far?

$$Q(s, a) \leftarrow \frac{1}{t} \sum_{t \text{ samples}} \left(r + \gamma \max_{a'} Q(s', a') \right)$$

Q-iteration -> Q-learning

- Running average

- Running average of t samples of a quantity x :

$$\begin{aligned}\bar{x}_t &= \frac{x_1 + x_2 + \dots x_{t-1} + x_t}{t} \\ &= \frac{x_1 + x_2 + \dots x_{t-1}}{t} \cdot \frac{(t-1)}{(t-1)} + \frac{x_t}{t} \\ &= \frac{(t-1)}{t} \bar{x}_{t-1} + \frac{1}{t} x_t \\ &= (1 - \alpha) \bar{x}_{t-1} + \alpha x_t \quad \text{where } \alpha = 1/t\end{aligned}$$

- Running average of Q:

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a'))$$

Monte Carlo Methods

- Model-free and it does not require a complete knowledge of the environment's dynamics (i.e., the state transition probabilities and rewards). Instead, **they learn directly from episodes of experience**.
- only after a **complete episode**, values are updated (Only can be used in **episodic problems**)
- There are a few strategies to be applied for continuing (non-episodic) environments. (e.g. artificial termination)

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:

$Q(s, a) \leftarrow \text{arbitrary}$

$Returns(s, a) \leftarrow \text{empty list}$

$\pi(a|s) \leftarrow \text{an arbitrary } \epsilon\text{-soft policy}$

Repeat forever:

(a) Generate an episode using π

(b) For each pair s, a appearing in the episode:

$G \leftarrow \text{return following the first occurrence of } s, a$

Append G to $Returns(s, a)$

$Q(s, a) \leftarrow \text{average}(Returns(s, a))$

(c) For each s in the episode:

$a^* \leftarrow \arg \max_a Q(s, a)$

For all $a \in \mathcal{A}(s)$:

$$\pi(a|s) \leftarrow \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(s)| & \text{if } a = a^* \\ \epsilon/|\mathcal{A}(s)| & \text{if } a \neq a^* \end{cases}$$

An on-policy first-visit MC control algorithm for ϵ -soft policies

ϵ -greedy (epsilon-greedy)

- Exploration and Exploitation:
 - **Exploration**: The agent tries new actions to discover more information about the environment.
 - **Exploitation**: The agent uses known information to maximize rewards, typically by choosing the best-known action.
- How ϵ -greedy Works: at each time step, the agent selects an action based on:
 - With probability ϵ , the agent selects an action at random (exploration).
 - With probability $1-\epsilon$, the agent selects the best-known action (exploitation).
- Most of the time we choose an action that has the maximal estimated action value, but with probability ϵ , we instead select an action at random.

$$\text{Prob(action)} = \frac{1 - \epsilon + \epsilon/|\mathcal{A}(s)|}{\epsilon/|\mathcal{A}(s)|}$$

← greedy action (minimal probability)

← non-greedy action

- In continuous action spaces, different strategies and algorithms are used:

Temporal Difference (TD) Learning

- **Model-free**,
- **Monte Carlo** (learn directly from raw experience) + **Dynamic Programming** (update estimates every time step)
- RL problems with discrete actions can be modeled as MDP, but the agent initially has no idea what the transition probabilities are (**does not know $T(s, a, s')$** and **$R(s, a, s')$**).
- It must experience each state and each transition at least once to know the rewards, and it must experience them multiple times if it is to have a reasonable estimate of the transition probabilities.

Temporal Difference (TD) Learning

- **What does TD(Time Difference) mean?**

- It means the differences between predicted (expected) true value and current estimation value. (not refer to the time between successive iterations of a learning algorithm)
- The predicted true value is obtained from the assumption that the Q-values for all other states (s') are optimal. (we use this to find a better estimated (close to optimal) value at state s .)

1. **Temporal Difference (TD):** In reinforcement learning, the term "temporal difference" refers to the discrepancy or difference between the expected value (or Q-value) of a state or action at a certain time step and the actual observed reward received at that time step.

2. **TD Error:** The TD error is a key concept in TD learning. It's the difference between the estimated value of a state (or action) and the observed reward plus the estimated value of the next state:

$$TD_error = \underline{R + \gamma \cdot Q(S', A')} - \underline{Q(S, A)}$$

Where:

- R is the observed reward.
- γ is the discount factor.
- $Q(S, A)$ is the estimated value of taking action A in state S .
- $Q(S', A')$ is the estimated value of the next state-action pair.

The TD error is used to update the value estimate of the current state-action pair.

Temporal Difference (TD) Learning

- What does TD(Time Difference) mean?

3. **TD Learning:** TD learning algorithms, like SARSA and Q-learning, update their value estimates based on these TD errors. They learn from the temporal differences observed during interaction with the environment.

For example, in Q-learning, the Q-value of a state-action pair is updated as follows:

$$Q(S, A) \leftarrow Q(S, A) + \alpha \cdot (R + \gamma \cdot \max_{a'} Q(S', a') - Q(S, A))$$

Where:

- α is the learning rate.

(*) using the observed reward and the maximum Q-value of the next state over all possible actions.

For SARSA:

$$Q(S, A) \leftarrow Q(S, A) + \alpha \cdot (R + \gamma \cdot Q(S', A') - Q(S, A))$$

(*) using the observed reward and the Q-value of the next state-action pair, which is chosen based on the current policy.

TD Learning Algorithm

- **Temporal Difference Learning (TD)**

- Similar to Value Iteration algorithm, but, take into account the fact that the agent has only partial knowledge of the MDP. (In general we assume that the agent initially knows only the possible states and actions, and nothing more.)
- The agent uses an exploration policy (for example, random policy) to explore the MDP, and progresses TD algorithm to update the estimates based on the transitions and rewards that are actually observed.

$$V_{k+1}(s) \leftarrow (1 - \alpha)V_k(s) + \alpha(r + \gamma \cdot V_k(s'))$$

- α is the learning rate (e.g., 0.01).

For each state s , it simply keep track of a running average of the rewards (immediate + future) at each state s

- Many similarities with Stochastic Gradient Descent (SGD):
 - handles one sample at a time
 - converge if you gradually reduce the learning rate (otherwise it will keep bouncing around the optimum Q-value)

Q-Learning algorithm

- **Q-Learning**

- Adaptation of the Q-value iteration with **no** information about the transition probabilities and the rewards
- It works by watching an agent play (e.g. randomly) and **gradually improving its estimates** of the Q-values.
- The optimal policy is choosing the action of the highest Q-value (e.g. greedy policy)

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

- For each state-action pair (s, a) , this algorithm **keeps track of a running average of the rewards** (immediate + future)
- Since the target policy would act optimally, **we take the maximum of the Q-Value estimates for the next state.**

SARSA algorithm

- **SARSA (State-Action-Reward-State-Action)**

- Update is done based on:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]$$

- This update is done after every transition from a nonterminal state S_t . If S_{t+1} is terminal, then $Q(S_{t+1}, A_{t+1})$ is defined as zero.
- This rule uses every element of the quintuple of events, $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, that make up a transition from one state action pair to the next.

SARSA vs. Q-Learning

- Q-Learning (off-policy)

Considers the best possible case

$$Q_{k+1}(s,a) \rightarrow Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

```
Q[s,a] = Q[s,a] + alpha*(reward - Q[s,a]
           + gamma*np.max([Q[s_,a_] for a_ in range(possible_action)]))
```

- SARSA (on-policy)

follow the current policy

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]$$

```
a_ = max_action(s_) if rand < (1-epsilon) else env.action_space.sample()
Q[s,a] = Q[s,a] + alpha*(reward - Q[s,a] + gamma*Q[s_,a_])
```

ε-greedy policy

SARSA vs. Q-Learning

- Policy:
 - ϵ -greedy is applied to balance the exploration and exploitation of reinforcement learning.
- Both are updated using the TD learning formula.
- Difference:
 - **Q-Learning (off-policy):** when passing the reward from the next state (s' , a') to the current state, it takes the maximum possible reward of the new state (s') and ignores whatever policy we are using. (the agent receives no policy)
 - **SARSA (on-policy):** we still follow the policy (ϵ -greedy), compute the next state (s'), and pass the reward corresponding to that action a' . (the agent is given the policy)
 - Hence, if our policy is greedy, SARSA and QL will be the same. But we are using ϵ -greedy normally, so there is a slight difference.

SARSA vs. Q-Learning

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

```
Initialize  $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$ , arbitrarily, and  $Q(\text{terminal-state}, \cdot) = 0$ 
Repeat (for each episode):
  Initialize  $S$ 
  Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
  Repeat (for each step of episode):
    Take action  $A$ , observe  $R, S'$ 
    Choose  $A'$  from  $S'$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
     $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$ 
     $S \leftarrow S'; A \leftarrow A';$ 
  until  $S$  is terminal
```

episode termination:

- goal reached
- failure condition
- time limit exceeded
- Environment-specific condition
- others

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

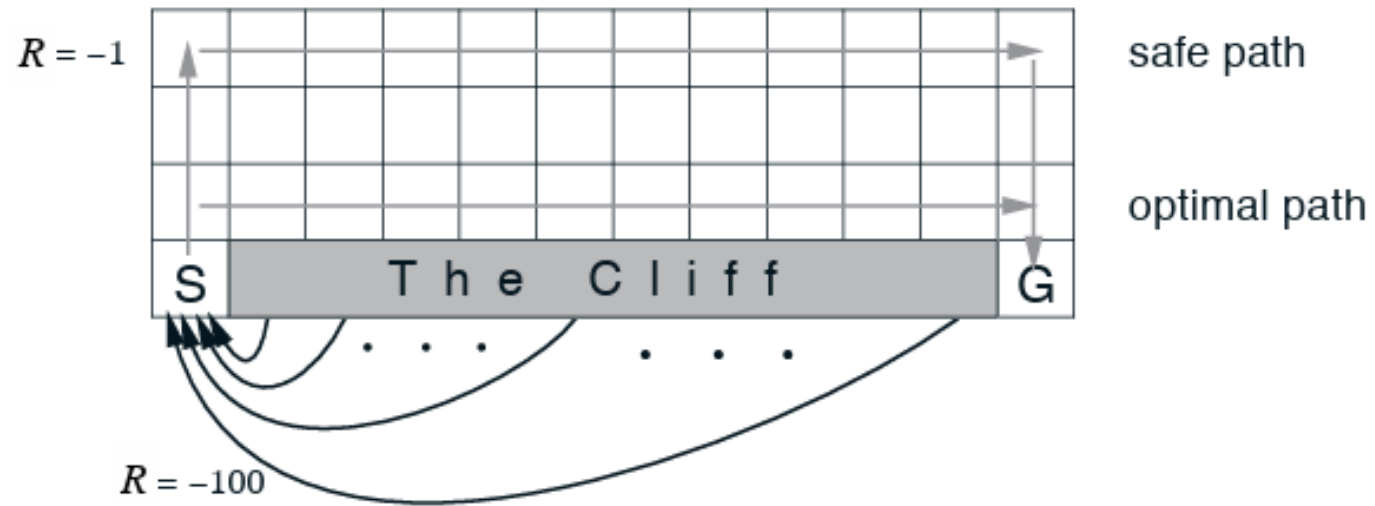
```
Initialize  $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$ , arbitrarily, and  $Q(\text{terminal-state}, \cdot) = 0$ 
Repeat (for each episode):
  Initialize  $S$ 
  Repeat (for each step of episode):
    Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
    Take action  $A$ , observe  $R, S'$ 
     $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$ 
     $S \leftarrow S'$ 
  until  $S$  is terminal
```

SARSA vs. Q-Learning

- **Some notes:**
 - Both approach work in a finite environment (or a discretized continuous environment)
 - QL directly learns the optimal policy while SARSA learns a “near” optimal policy. QL is a more aggressive agent, while SARSA is more conservative.
 - An example is walking near the cliff. **QL will take the shortest path** because it is optimal (with the risk of falling), while **SARSA will take the longer, safer route** (to avoid unexpected falling).
 - In practice, if you want to fast in a fast-iterating environment, QL should be your choice. However, if mistakes are costly (unexpected minimal failure — robots), then SARSA is the better option.
 - If your state space is too large, try exploring the deep Q-network.

SARSA vs. Q-Learning

- The cliff-walking example (Sutton and Barto)



On-policy and Off-policy

- **On-Policy Methods:**

- the agent learns the value of the policy it is currently following. This means the agent learns about and from the very policy it uses to make decisions.
- Behavior policy = target policy (decision-making policy = learning policy)
- SARSA

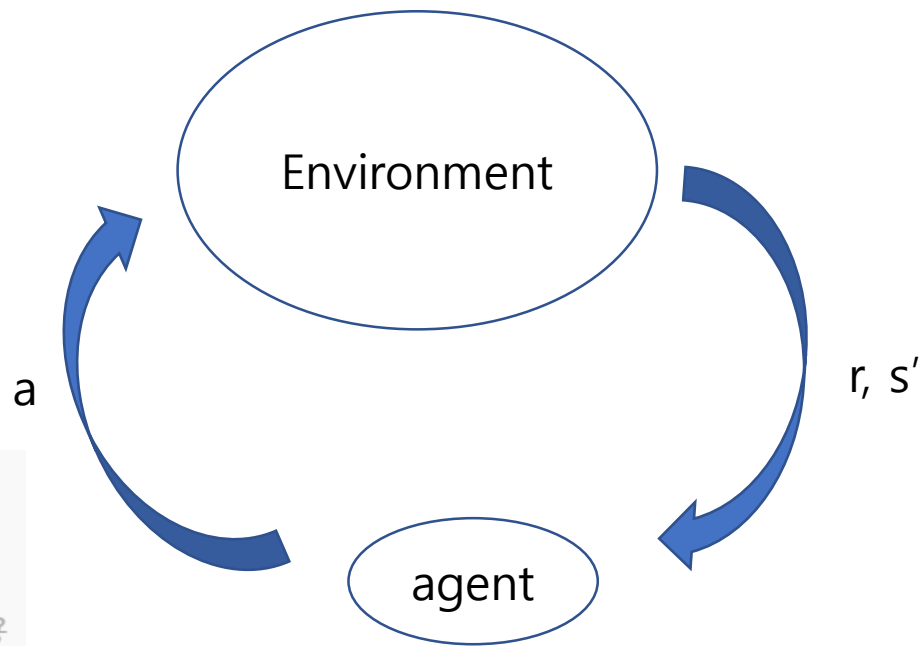
- **Off-Policy Methods:**

- It allows the agent to learn about one policy (the target policy) while following another policy (the behavior policy). The agent learns from actions that are outside its current strategy.
- Behavior policy $\neq$ target policy (decision-making policy $\neq$ learning policy)
- Q-learning, expected SARSA

On-policy and Off-policy

behavioral policy : select a (e-greedy)

```
if random.random() < epsilon:  
    next_action = random.choice(actions) # 탐험  
else:  
    next_action = max(actions, key=lambda a: q_values[(state, a)]) # 활용
```



$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

target policy : select a' (max or current policy)

Approximate Q-Learning (DQN)

- **Problem in Q-Learning**

- It does not scale well to large MDPs with **many states** and **actions**.
- The solution is to find a function that approximates the Q-values using some parameters. (Approximate Q-Learning)
- Uses DQN(deep Q-network) to estimate Q-Values (Deep Q-Learning)

- **How to train DQN?**

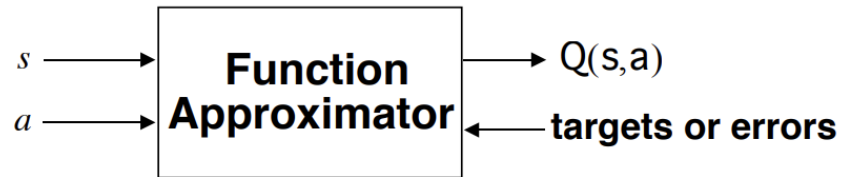
- Thanks to Bellman, we want the approximate Q-value to be as close as to the reward r (that we actually observe after playing action a in state s), plus the discounted value of playing optimally from then on.
- Target Q-value:

$$Q_{\text{target}}(s, a) = r + \gamma \cdot \max_{a'} Q_{\theta}(s', a')$$

when we update the value at state s , **we're making the assumption that the values at all other states s' are accurate or optimal.**

Approximate Q-Learning (DQN)

General Idea



$$\text{Loss} = (\text{Target} - f(s,a,w))^2$$
$$d\text{Loss}/dw = -(Target - f) * df/dw$$
$$w = w - \alpha * d\text{Loss}/dw$$

Could be:

- table
 - Backprop Neural Network
 - Radial-Basis-Function Network
 - Tile Coding (CMAC)
 - Nearest Neighbor, Memory Based
 - Decision Tree
- gradient-descent methods

$$Q(s,a) = f(s,a,w)$$

weight vector

e.g., gradient-descent Sarsa:

$$w \leftarrow w + \alpha \left[\underbrace{r_{t+1} + \gamma Q(s_{t+1}, a_{t+1})}_{\text{target value}} - \underbrace{Q(s_t, a_t)}_{\text{estimated value}} \right] \nabla_w f(s_t, a_t, w)$$

target value

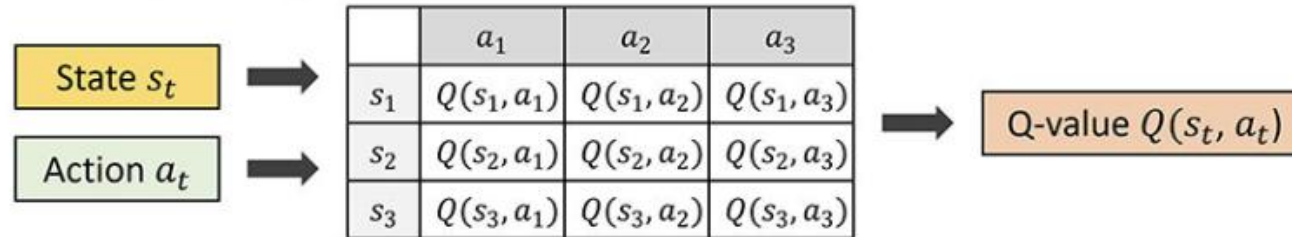
estimated value

standard
backprop
gradient

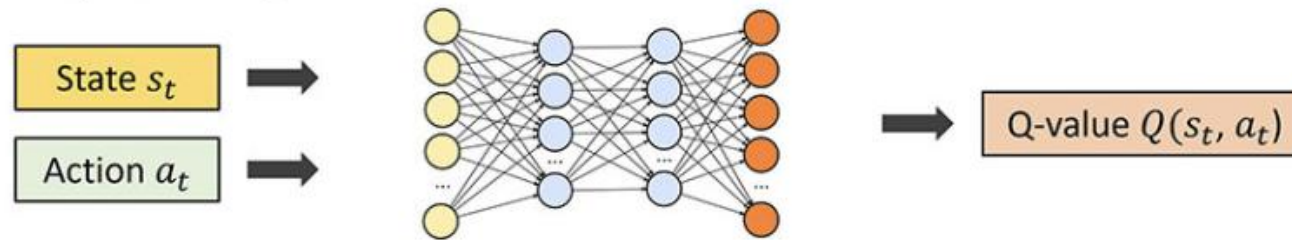
Approximate Q-Learning (DQN)

- Neural network: estimate a probability for each action
 - Input: an observation
 - Output: action to be taken

Classic Q-learning



Deep Q-learning



DQN and GD

Gradient Descent: 기본 개념

- 목적: 어떤 함수 $f(\theta)$ 를 최소화(또는 최대화)하기 위해, 함수의 기울기(gradient)를 사용하여 파라미터 θ 를 반복적으로 업데이트합니다.
- 업데이트 공식:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} f(\theta)$$

- α : 학습률(learning rate).
- $\nabla_{\theta} f(\theta)$: 파라미터 θ 에 대한 손실 함수의 기울기.

핵심 아이디어:

- 기울기가 가리키는 방향으로 이동하여 $f(\theta)$ 를 점진적으로 최소화합니다.

DQN과 Gradient Descent의 차이점

- **Gradient Descent**는 고정된 손실 함수와 데이터셋에서 작동하는 반면, **DQN**은 환경에서 동적으로 데이터를 수집하며 손실 함수를 업데이트합니다.
- DQN은 강화 학습의 특성상 학습 목표(타겟 Q-값)가 시간에 따라 변화하며, 이는 일반적인 Gradient Descent와의 주요 차이점입니다.

DQN: Q-Learning + Neural Network

- 목적: 상태-행동 가치 함수 $Q(s, a)$ 를 근사하여, 최적 정책을 학습합니다.
- DQN의 역할:
 - Q-Learning에서 테이블 대신 신경망을 사용하여 $Q(s, a)$ 를 근사.
 - 손실 함수와 Gradient Descent를 사용하여 신경망 파라미터를 업데이트.

DQN 손실 함수:

$$L(\theta) = \mathbb{E} \left[\left(y - Q(s, a; \theta) \right)^2 \right]$$

- $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$: 타겟 Q-값.
 - r : 보상.
 - γ : 할인율(discount factor).
 - θ^- : 타겟 네트워크의 파라미터.
- $Q(s, a; \theta)$: 현재 네트워크의 Q-값.

파라미터 업데이트:

DQN은 Gradient Descent를 사용하여 손실 함수 $L(\theta)$ 를 최소화:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L(\theta)$$

DQN

- **Experience (replay) Memory**

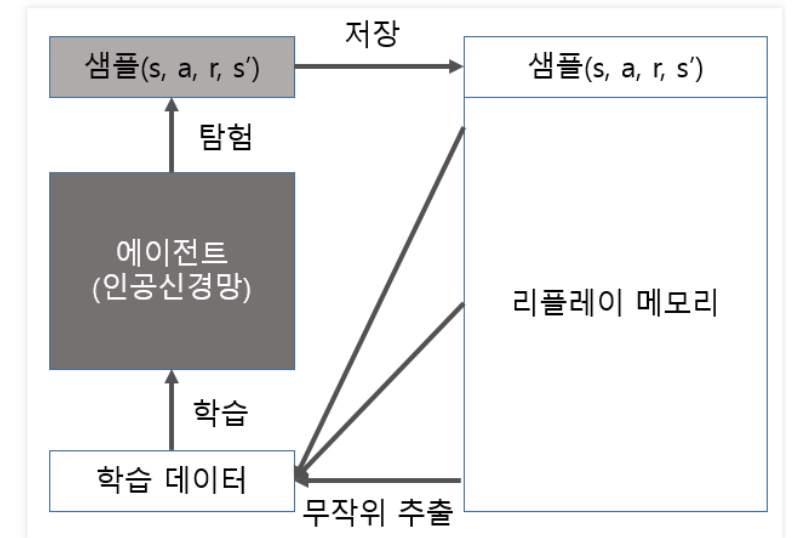
- Stores experiences including state transitions, rewards and actions (that are necessary to perform Q learning)
- Makes mini-batches to update neural networks.
- Merits:
 - Reduces correlation between experiences in updating DNN
 - Increases learning speed with mini-batches
 - Reuses past transitions to avoid catastrophic forgetting

- **Two Q-Networks**

- Use two separate Q-networks (**Action Network** and **Target Network**)
- Periodically update (by copying Action Network to Target Network)

- **Gradient Clipping**

- forces the gradient values (element-wise) to a specific minimum or maximum value (+1 and -1)



Deep Q-learning

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

With probability ε select a random action a_t
otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

Execute action a_t in emulator and observe reward r_t and image x_{t+1}

Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

Every C steps reset $\hat{Q} = Q$

End For

End For

preprocess example: screen display interface for games

ε -greedy

Example code:

- Note that we do not clear out the memory after each episode, this enables us to recall and build batches of experiences from across episodes.
- <https://towardsdatascience.com/deep-q-network-dqn-ii-b6bf911b6b2c>

```
Initialize network  $Q$ 
Initialize target network  $\hat{Q}$ 
Initialize experience replay memory  $D$ 
Initialize the Agent to interact with the Environment
while not converged do
    /* Sample phase
     $\epsilon \leftarrow$  setting new epsilon with  $\epsilon$ -decay
    Choose an action  $a$  from state  $s$  using policy  $\epsilon$ -greedy( $Q$ )
    Agent takes action  $a$ , observe reward  $r$ , and next state  $s'$ 
    Store transition  $(s, a, r, s', done)$  in the experience replay memory  $D$ 

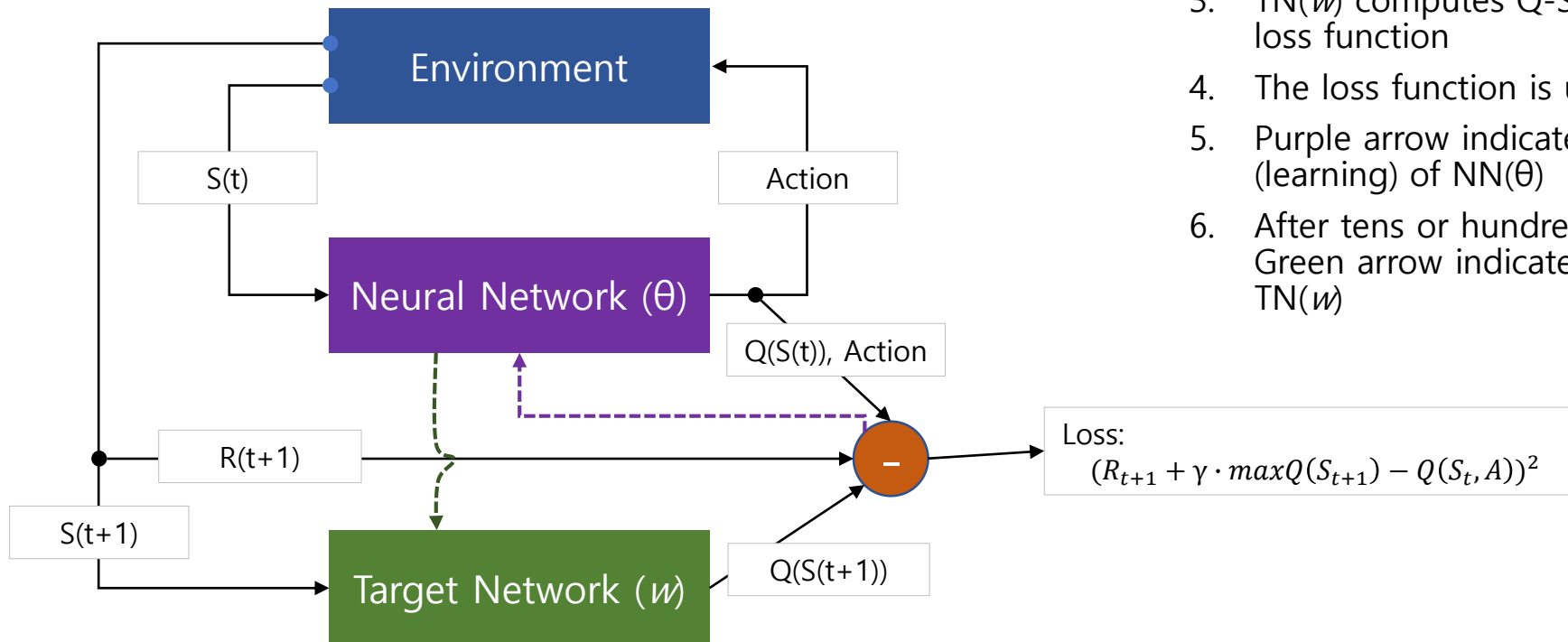
    if enough experiences in  $D$  then
        /* Learn phase
        Sample a random minibatch of  $N$  transitions from  $D$ 
        for every transition  $(s_i, a_i, r_i, s'_i, done_i)$  in minibatch do
            if  $done_i$  then
                |  $y_i = r_i$ 
            else
                |  $y_i = r_i + \gamma \max_{a' \in \mathcal{A}} \hat{Q}(s'_i, a')$ 
            end
        end
        end
        Calculate the loss  $\mathcal{L} = 1/N \sum_{i=0}^{N-1} (Q(s_i, a_i) - y_i)^2$ 
        Update  $Q$  using the SGD algorithm by minimizing the loss  $\mathcal{L}$ 
        Every  $C$  steps, copy weights from  $Q$  to  $\hat{Q}$ 
    end
end
```

Deep Q-learning

- **General flow of Deep Q-Learning**

- Deep Q-Network (DQN) Perspective

1. Input $S(t)$ to $NN(\theta)$ then outputs an action
2. From the Action, Environment gives $S(t+1)$ and $R(t+1)$
3. $TN(\omega)$ computes $Q(S(t+1))$, then computes the loss function
4. The loss function is used to train $NN(\theta)$
5. Purple arrow indicates the parameter update (learning) of $NN(\theta)$
6. After tens or hundreds of $NN(\theta)$ update, Green arrow indicates parameters copy to $TN(\omega)$



DDQN

- Q-learning methods are known to be overestimating the Q value
 - DQN and other Q learning methods have this common issue and their performances are lowered due to that.
 - DDQN reduces overestimation by decoupling the **action selection** and **action evaluation** processes. Instead of using the same Q-network for both, DDQN uses the **main Q-network** to select the best action and the **target Q-network** to evaluate its value.

The target value in DDQN is calculated as:

$$y = r + \gamma Q_{\text{target}}(s', \arg \max_{a'} Q_{\text{main}}(s', a'))$$

Key differences:

1. **Action selection** ($\arg \max$) is performed using the main Q-network.
2. **Action evaluation** is performed using the target Q-network.

In standard DQN, the target Q-value is calculated as:

$$y = r + \gamma \max_{a'} Q_{\text{target}}(s', a')$$